

CD 租售店线程同步实验报告

1. 实验目的

- 理解多线程并发访问共享资源时的竞态问题（库存数量、租借状态、日志记录）。
- 掌握 `synchronized`、`wait/notifyAll` 等 Java 基本同步机制的用法与适用场景。
- 能够设计“进货 / 销售 / 租借”并发协作模型，并保证程序能在限定时间内安全停止并输出日志。

2. 实验内容与要求概述

本实验模拟一个 CD 租售店，包含两类资源：

- **可售 CD**：10 种，每种初始 10 张；销售线程随机购买 1~5 张。
- **可租 CD**：10 张（编号 1~10）；租借线程随机租借某一张，若已出租则等待或放弃；借到后随机等待 200~300ms 归还。

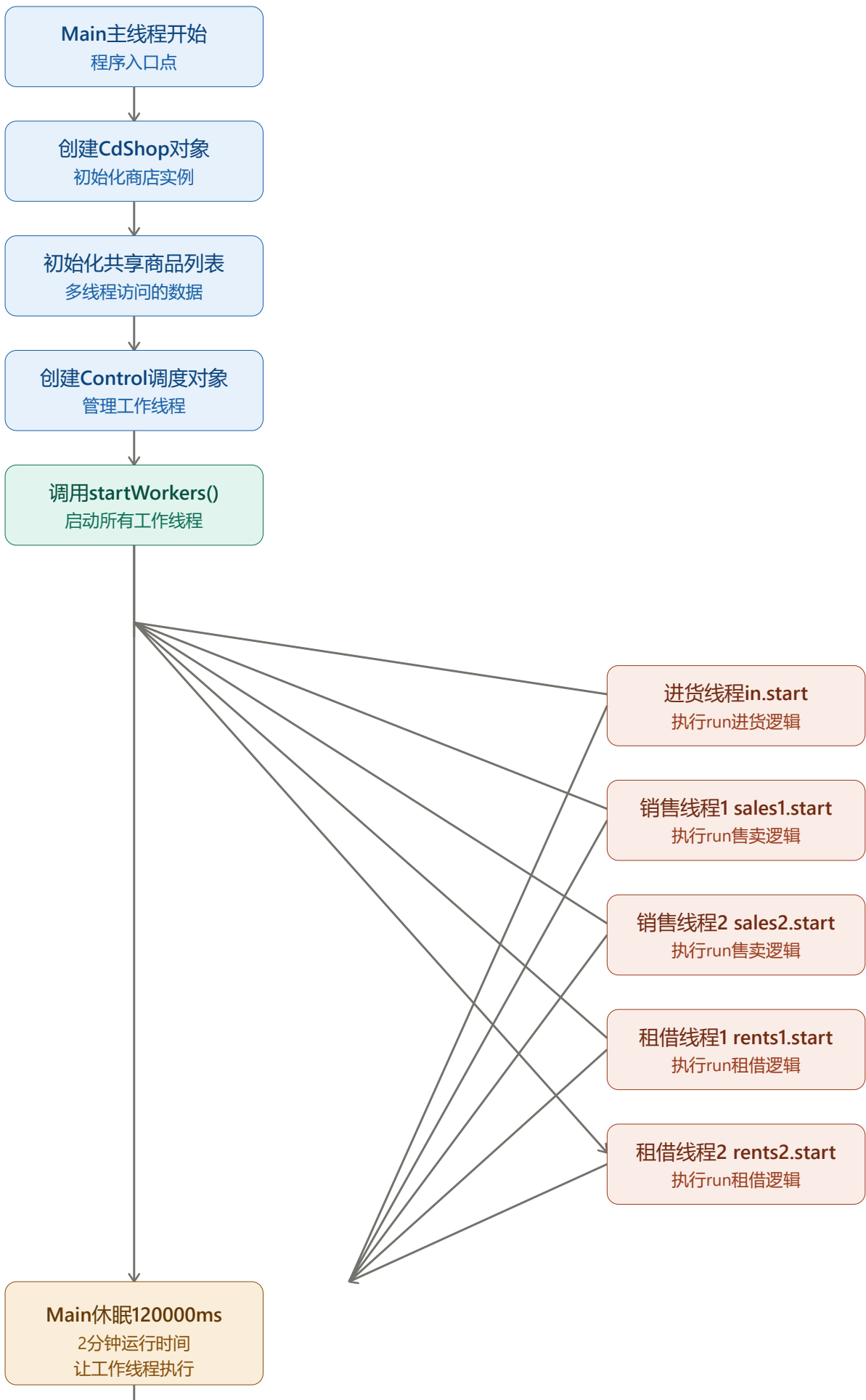
并发线程要求：

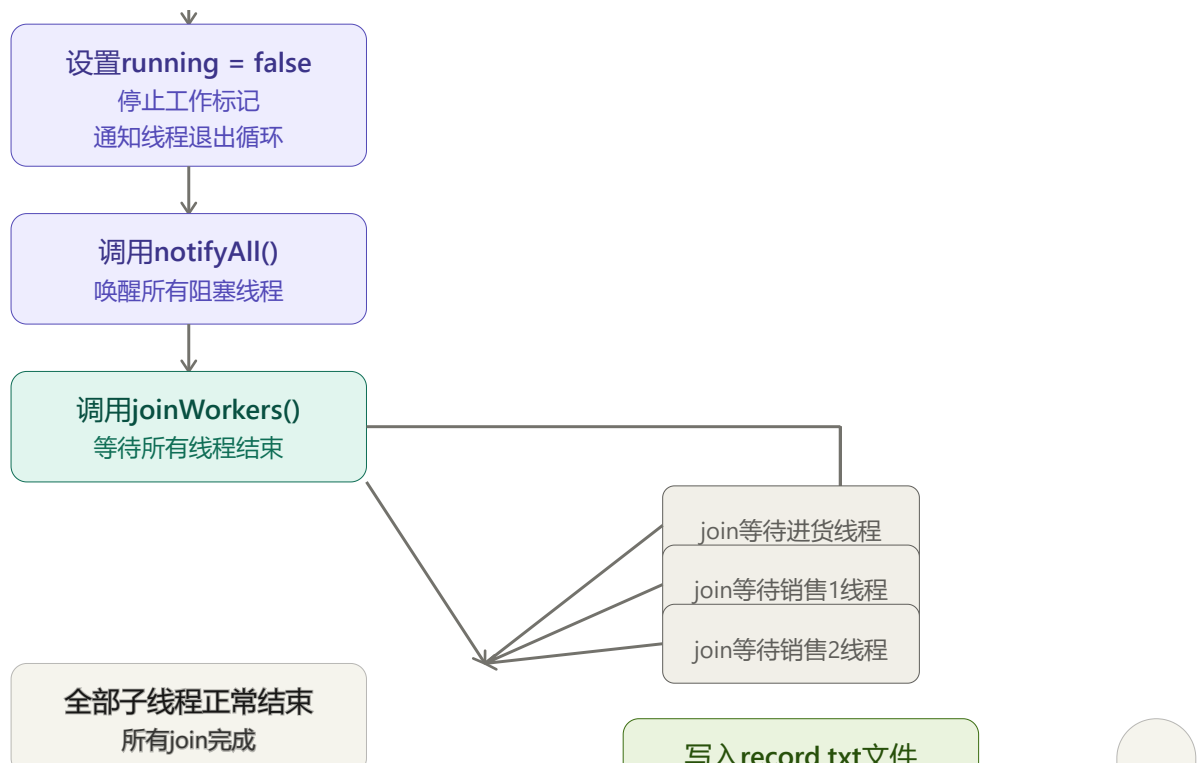
- **进货线程**：仅 1 个；每 1 秒补满所有可售 CD（不足 10 则补到 10）。
- **销售线程**：2 个或以上；启动间隔为 200ms 以内随机；库存不足则随机选择等待或放弃。
- **租借线程**：2 个或以上；启动间隔为 200ms 以内随机；CD 已出租则随机选择等待或放弃。

程序运行 2 分钟后停止，输出全部行为记录到 `record.txt`，并要求连续运行两次输出两段记录。

实现代码位于：[CDShop.java](#)

3. 总体实现思路





整体采用“共享资源 + 多工作线程 + 协作同步”的模型：

- 将可售库存列表 `salelist`、可租列表 `rentlist` 作为共享资源。
- 进货线程周期性补货；销售线程在库存不足时通过 `wait()` 进入等待，补货后通过 `notifyAll()` 唤醒重试。
- 租借线程对每张可租 CD 的“已借出/已归还”状态进行同步控制；已借出时可等待归还或放弃。
- 运行控制使用 `volatile boolean running` 作为停止开关；主线程在 2 分钟后置 `running=false`，并对两个资源锁对象分别 `notifyAll()`，避免工作线程永久等待。
- 所有动作调用 `addRecord()` 记录时间戳与行为字符串，最终写入文件。
-

4. 主要类与数据结构说明

4.1 CD / SaleCD / RentCD

- `CD`：仅包含 `String name`，作为公共父类。
- `SaleCD`：
 - `count`：库存数量。
 - 构造时将 `count=10`，并用静态自增 `id` 生成 `cd1~cd10` 名称。
- `RentCD`：
 - `ID`：编号 1~10。
 - `status`：是否已借出（`true` 表示已借出）。

4.2 CDShop（共享资源容器）

- `List<SaleCD> salelist`：可售 CD 列表。
- `List<RentCD> rentlist`：可租 CD 列表。
- `volatile boolean running=true`：运行开关。
- `List<String> records`：日志列表，使用 `Collections.synchronizedList` 包装以保证并发 `add` 的安全性。
- `addRecord(String action)`：记录当前 `LocalTime` + 行为描述。

4.3 Control（线程管理器）

- 统一创建并 `start()` 5 个工作线程（1 进货、2 销售、2 租借）。
- 主线程停止后统一 `join()` 等待所有线程结束，确保日志完整。

5. 线程行为设计

5.1 进货线程 InThread

- 以 `salelist` 作为同步锁对象：`synchronized (cdshop.salelist) { ... }`
- 循环期间遍历所有 `SaleCD`，将不足 10 的库存补到 10，并记录日志。
- 每轮补货结束后 `notifyAll()` 唤醒可能等待库存的销售线程。
- 通过 `wait(1000)` 实现“每秒检查一次”的节拍，同时释放锁让销售线程可以进入临界区。
- 结束前额外 `notifyAll()`，防止停止时仍有销售线程在等待。

5.2 销售线程 SaleThread

- 启动间隔：每次循环先 `sleep(random 0~199ms)` 模拟随机启动/到店购买。
- 随机挑选一种 CD，并生成购买数量 `count=1~5`。
- 以 `salelist` 作为同步锁对象，保证以下操作原子性：
 - 检查库存是否足够
 - 不足则随机选择等待或放弃
 - 足够则扣减库存并记录
- 库存不足时的协作逻辑：
 - 先 `notifyAll()` 提醒进货线程（如果它正在 `wait(1000)`，会被提前唤醒进行补货）。
 - 使用 `while (running && cd.count < count)` 循环判断并 `wait()`，避免虚假唤醒导致逻辑错误。
 - 等待后再次判断库存，满足条件才扣减。
- 通过带标签的 `continue outer` 实现“放弃购买后立即进入下一轮随机行为”。

5.3 租借线程 RentThread

- 启动间隔：每次循环先 `sleep(random 0~199ms)`。
- 随机挑选一张 `RentCD`。
- 以 `rentlist` 作为同步锁对象，保证以下操作原子性：
 - 判断该 CD 是否已被借走 (`status`)
 - 已借走则随机等待归还或放弃
 - 可借则将 `status=true` 并记录借出
- 归还流程：
 - 借到 CD 后线程在临界区外 `sleep(200~300ms)` 模拟租借时长。
 - 随后再次进入 `synchronized (rentlist)`，将 `status=false`，记录归还并 `notifyAll()` 唤醒等待该资源的租借线程。

6. 同步方式与正确性说明

本实验采用的同步方式为：**内置锁 (monitor) + 条件等待 (wait) + 条件通知 (notifyAll)**，并通过 `volatile` 保证停止信号可见性。

6.1 锁对象选择

- 销售/进货共享同一把锁：`cdshop.salelist`
 - 原因：两类线程都围绕“可售库存数量”协作，必须在同一监视器上执行 `wait/notifyAll` 才能配合。
- 租借共享一把锁：`cdshop.rentlist`
 - 原因：租借线程围绕“可租 CD 的借出状态”协作，采用同一监视器实现等待与唤醒。

6.2 wait/notifyAll 的使用规则

- `wait()` / `wait(timeout)` 必须在 `synchronized(锁对象)` 内调用，且会：
 - 释放该锁
 - 让线程进入该锁的等待队列
 - 被 `notify/notifyAll` 或超时后重新竞争锁返回
- 使用 `while(条件不满足) wait()`：
 - 防止虚假唤醒 (spurious wakeup)
 - 防止被唤醒时条件已被其它线程再次改变

本实现中：

- 销售线程在 `cd.count < count` 时等待，直到进货线程补货并 `notifyAll()`。
- 租借线程在 `cd.status==true` 时等待，直到归还线程 `notifyAll()`。

6.3 running 的停止与唤醒策略

- `running` 声明为 `volatile`，保证主线程写入 `false` 对工作线程立刻可见。
- 主线程停止时，分别对 `salelist` 与 `rentlist` 执行一次 `notifyAll()`：
 - 目的：唤醒可能正在 `wait()` 的销售/租借/进货线程，使其有机会检测 `running` 并退出循环，避免死等。

6.4 日志同步

- `records` 使用 `Collections.synchronizedList(new ArrayList<>())` 包装：
 - 使得 `records.add(...)` 操作线程安全。
- 写文件时对 `cdshop.records` 进行同步遍历：
 - 避免遍历期间日志仍在被追加造成并发问题（尽管此时线程已 `join` 完成，仍保持同步习惯）。

7. 程序运行流程与输出

- `main` 中创建 `PrintWriter("record.txt")`，调用两次 `runOnce(1/2, writer)`，得到两段运行记录。
- 每次 `runOnce`：
 - 初始化 10 种可售 CD（每种 10 张）、10 张可租 CD（编号 1~10）。
 - 启动 5 个线程并运行 120000ms（2 分钟）。
 - 设置 `running=false` 并唤醒等待线程。
 - `join` 等待所有线程退出后，将 `records` 写入文件，形成：
 - `=====第X次运行=====`
 - 之后为若干行 `时间戳 + 行为描述`

8. 总结与可改进点

- 本实现用最基础的 `monitor` 模型完成了库存补货与销售等待、租借与归还等待两套条件同步，结构清晰，符合实验对 `wait/notifyAll` 的训练目标。
- 可进一步优化的方向：
 - 使用更细粒度锁（例如对单个 `SaleCD` 或单个 `RentCD` 加锁）提高并发度。
 - 使用 `ReentrantLock + Condition` 将“库存不足”“已被借走”等条件表达得更直观，并可做更精细的唤醒。
 - 为日志加上线程名/线程类型字段，便于分析并发行为（当前已通过前缀如 `sale:/rent:/in:` 区分）。

核心代码

```
package thread;
```

```

import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.stream.Collectors;
import java.util.stream.Stream;
import java.time.LocalDateTime;
import java.util.Collections;
import java.io.PrintWriter;
import java.io.IOException;

class CD {
    String name;
}

//可售cd列表
class SaleCD extends CD {
    static int id = 1;

    SaleCD() {
        name = "cd" + (id++);
        count = 10;
    }

    int count;
}

//可租cd列表
class RentCD extends CD {
    int ID;
    boolean status=false;//true booked

    RentCD(int ID) {
        this.ID = ID;
        this.name = "rent" + ID;
    }
}

//进货线程
class InThread extends Thread {
    CDShop cdshop;

    InThread(CDShop cdshop) {
        this.cdshop=cdshop;
    }

    public void run() {
        synchronized (cdshop.salelist) {
            while(cdshop.running){

```



```

        for (SaleCD cd : cdshop.salelist) {
            if(cd.count<10){
                cd.count=10;
                cdshop.addRecord("in: curr cd " + cd.name + " 已补满");
            }
        }
        cdshop.salelist.notifyAll();
        try {
            cdshop.salelist.wait(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    cdshop.salelist.notifyAll();
}
}

//销售线程
class SaleThread extends Thread {
    CDShop cdshop;
    static Random r = new Random();

    SaleThread(CDShop cdshop) {
        this.cdshop=cdshop;
    }

    public void run() {
        //加锁或者同步
        outter:
        while (cdshop.running) {
            int sleepTime=r.nextInt(200);
            try {
                Thread.sleep(sleepTime);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            SaleCD cd = cdshop.salelist.get(r.nextInt(cdshop.salelist.size()));
            int count = r.nextInt(5)+1;//购买数量为5以内的随机数1~5
            synchronized (cdshop.salelist) {

                //通知进货线程
                if (cd.count < count) {
                    //当前这种cd还剩count张
                    cdshop.addRecord("sale: curr cd " + cd.name + " 数量不足 ");
                    cdshop.salelist.notifyAll();
                }
                boolean ifwait=false;
                while (cdshop.running && cd.count < count) {
                    if (r.nextBoolean()) {
                        cdshop.addRecord("sale: 放弃购买 " + cd.name);
                        continue outter;
                    }
                }
            }
        }
    }
}

```

```

        } else {
            ifwait=true;
            try {
                cdshop.addRecord("sale: 等候" + cd.name + " 进货 ");
                //System.out.println("sale: 等候" + cd.name + "进货 ");
                //等待进货线程进货完成
                cdshop.salelist.wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
    if (!cdshop.running) {
        break outter;
    }
    cd.count -= count;
    if(!ifwait)
        //没经过等待就卖了
        cdshop.addRecord("sale: curr cd " + cd.name + " 销售" + count + " 剩余 "
+ cd.count);
    else
        //经过等待后才卖了
        cdshop.addRecord("sale: curr cd " + cd.name + " 等候后销售" + count + "
剩余 " + cd.count);
    }
}
}

```

// 租借线程：可以有两个或两个以上，租借CD店的可租借CD，启动时间为200ms以内的随机数。租借序号为1-10随机序号的CD，如果该CD已经出租则随机选择等候或者放弃。如果可以借到CD则随机等候200~300ms然后归还。

```

class RentThread extends Thread {
    CDShop cdshop;
    static Random r=new Random();

    RentThread(CDShop cdshop) {
        this.cdshop=cdshop;
    }

    public void run(){
        outter:
        while(cdshop.running){
            int sleepTime=r.nextInt(200);
            try {
                Thread.sleep(sleepTime);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            RentCD cd = cdshop.rentlist.get(r.nextInt(cdshop.rentlist.size()));
            boolean ifwait = false;
            synchronized (cdshop.rentlist) {

```

```

        if(cd.status){
            cdshop.addRecord("rent: curr cd " + cd.ID + " 已被借走了 ");
            cdshop.rentlist.notifyAll();
        }
        while(cdshop.running && cd.status){
            if(r.nextBoolean()){
                ifwait = true;
                try {
                    cdshop.addRecord("rent: 等候" + cd.ID + "归还");
                    cdshop.rentlist.wait();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }else{
                cdshop.addRecord("rent: 放弃租借 " + cd.ID);
                continue outter;
            }
        }
        if (!cdshop.running) {
            break;
        }
        cd.status = true;
        if(!ifwait){
            //没经过等待就借借了
            cdshop.addRecord("rent: curr cd " + cd.ID + " 直接借了 ");
            //System.out.println("rent: curr cd" + cd.ID + " 直接借了 ");
        }else{
            //经过等待后才借借
            cdshop.addRecord("rent: curr cd " + cd.ID + " 等候后借借了 ");
            //System.out.println("rent: curr cd" + cd.ID + " 等候后借借了 ");
        }

        int waitTime=r.nextInt(101)+200;
        try {
            Thread.sleep(waitTime);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        synchronized (cdshop.rentlist) {
            cd.status=false;
            cdshop.addRecord("rent: curr cd " + cd.ID + " 已归还");
            cdshop.rentlist.notifyAll();
        }
    }
}

//一个线程管理器，统一start和统一join
class Control {
    CDShop cdshop;
    InThread in;
    SaleThread sales1;
}

```

```

SaleThread sales2;
RentThread rents1;
RentThread rents2;

Control(CDShop cdshop) {
    this.cdshop = cdshop;
}

public void startWorkers() {
    in = new InThread(cdshop);
    sales1 = new SaleThread(cdshop);
    sales2 = new SaleThread(cdshop);
    rents1 = new RentThread(cdshop);
    rents2 = new RentThread(cdshop);

    in.start();
    sales1.start();
    sales2.start();
    rents1.start();
    rents2.start();
}

public void joinWorkers() throws InterruptedException {
    in.join();
    sales1.join();
    sales2.join();
    rents1.join();
    rents2.join();
}
}

public class CDShop {
    List<SaleCD> salelist = new ArrayList<>();
    List<RentCD> rentlist=new ArrayList<>();
    volatile boolean running=true;

    //日志
    List<String> records = Collections.synchronizedList(new ArrayList<>());

    //记录方法
    void addRecord(String action){
        String time=LocalTime.now().toString();
        records.add(time + " " +action);
    }

    static void runOnce(int round,PrintWriter writer) throws InterruptedException{
        SaleCD.id = 1;
        CDShop cdshop = new CDShop();
        cdshop.salelist =
        Stream.generate(() -> new SaleCD())
                .limit(10)

```

```

        .collect(Collectors.toList());
cdshop.rentlist =
Stream.iterate(1, i -> i + 1)
    .limit(10)
    .map(RentCD::new)
    .collect(Collectors.toList());

Control control = new Control(cdshop);
control.startWorkers();
Thread.sleep(120000);
cdshop.running=false;
synchronized (cdshop.salelist) {
    cdshop.salelist.notifyAll();
}
synchronized (cdshop.rentlist) {
    cdshop.rentlist.notifyAll();
}
control.joinWorkers();

writer.println("=====第"+round+"次运行=====");
synchronized (cdshop.records) {
    for(String record:cdshop.records){
        writer.println(record);
    }
}
writer.println();
writer.flush();
}

// 主方法
public static void main(String[] args) throws Exception {
    try (PrintWriter writer = new PrintWriter("record.txt")) {
        runOnce(1,writer);
        runOnce(2,writer);
    }
}
}

```